

Overview

Track 16.4 —  LangGraph · 16-ai-tech-stack/tracks/langgraph/README.md

LangGraph — Stateful, Controllable Agent Orchestration

Where you are: Track 16.4 of the AI Tech Stack module. **Time:** ~10–12 hours · **Prereq:** Python, `../../../../06-ai-agents/` (the ReAct loop), and ideally `../langchain/` for messages and tools.

Every team that builds an agent writes the same `while` loop, then bolts on a step counter, then a retry, then a "stuff so far" dict, and then someone asks whether a human can approve the refund before it goes out — at which point the whole design collapses, because the run lives in one process's stack. LangGraph is that loop rebuilt as a **persisted state machine**: nodes do work, edges decide control flow, and the entire state is checkpointed after every step, so the run can pause for three hours, survive a deploy, be replayed from step 4, and be handed to an auditor.

Targets **LangGraph 1.x** (verified against 1.2.11). Every script here runs **offline, with no API key** — the models are deterministic Python functions, because the lesson is the graph.

Learning objectives

By the end of this track you can:

1. Explain why a DAG-of-prompts breaks once you need loops, branching, retries or approval — and draw the same workflow as a state machine instead.
2. Design a state schema: choose channels, pick reducers (`add_messages`, `operator.add`, custom), and predict exactly what happens when two parallel nodes write the same key.
3. Build a graph with conditional edges, a cycle, and **two** stopping conditions — a semantic exit and a budget in state — without relying on `recursion_limit`.
4. Implement a ReAct agent as an explicit graph, including tool errors returned to the model rather than raised, and parallel tool calls in one turn.
5. Add persistence (`InMemorySaver` → `SqliteSaver` → `PostgresSaver`), resume a thread in a fresh process, and time-travel to an earlier checkpoint to fork an alternate run.
6. Put a human approval gate in the middle of a run with `interrupt()` / `Command(resume=...)`, and say why every node above an interrupt must be idempotent.
7. Choose between a plain loop, `create_agent`, a hand-built `StateGraph`, and a general durable workflow engine — and defend the choice with something other than fashion.

Suggested path

#	Do this	File	Time
1	Read the core concepts	notes/01-core-concepts.md	1.5h
2	Run the fundamentals script	code/graph_basics.py	1h
3	Build the ReAct loop by hand	code/react_agent_from_scratch.py	1.5h
4	Read the production craft note	notes/02-production-craft.md	1.5h
5	Persistence, interrupts, time travel	code/checkpoint_and_interrupt.py	1.5h
6	Do the exercises	lab/EXERCISES.md	4h
7	Present it back	slides/ (langgraph.pptx)	0.5h
8	Read the primary sources	papers/PAPERS.md	1h

Install once:

```
/Users/srinip/ai-all/.venv/bin/pip install -r code/requirements.txt  
/Users/srinip/ai-all/.venv/bin/python code/graph_basics.py
```

The 16 terms you must own

StateGraph · channel · reducer · add_messages · superstep · conditional edge · Command · Send · subgraph · checkpointer · thread_id · checkpoint_id · interrupt · Command(resume=...) · durability · stream_mode

Exit check

Ship a single graph that a colleague can run offline and that demonstrates, in one trace: a cycle with a budget in state, a tool failure the agent recovers from, an `interrupt()` approval gate that survives the process exiting and restarting against a SQLite checkpointer, and a time-travelled fork that reaches a different outcome from the same checkpoint. Then answer, in two sentences, the only question that matters: **what would this cost you to rebuild as a plain `while` loop?**